

Passing an **Excel file from Logic Apps to your Hosted Agent via an HTTP payload** requires careful handling to avoid corrupting the file. In Azure Logic Apps, binary files like Excel .xlsx spreadsheets cannot be cleanly embedded directly into a standard text-based JSON body without encoding them first. [1]

The recommended pattern is to pass the file as **Base64-encoded text inside the JSON payload**. This keeps your architecture clean, native, and easy to parse inside your hosted agent's container application. [1, 2]



1. The Logic Apps Workflow (The Sender)

In your Logic App, you will pull the Excel file from a storage source (like SharePoint, OneDrive, or Azure Blob Storage) and send it over HTTP: [1, 3]

1. **Trigger:** e.g., *When a file is created in a folder*.
2. **Action:** *Get file content* (Set **Infer Content Type** to **No** to ensure you get the raw, unaltered byte array).
3. **Action:** **HTTP Action** (POSTing to your Hosted Agent's Endpoint). [1, 3, 4, 5]

The HTTP Action Configuration

Configure the JSON body of your HTTP action to pass both the filename and the encoded content: [6]

-
- **Method:** POST
- **URI:** https://<your-foundry-endpoint>/api/agent/process (Your Hosted Agent's API route)
- **Headers:** Content-Type: application/json
- **Body:**
-

```
{
  "fileName": "@triggerOutputs()?['body']?['Name']",
  "fileContentBase64":
"@base64(body('Get_file_content')?['$content'])"
}
```

⚠️ Crucial Logic Apps Detail: Wrapping the file content in the @base64() expression ensures that the binary data safely traverses the HTTP layer without missing character corruption. [1]

2. The Hosted Agent Code (The Receiver)

Because your agent is a custom application running on Foundry's hosted architecture, you can use frameworks like **FastAPI** or **Flask** to receive the request. You will catch the Base64 string, decode it back into raw bytes, and pass it directly to data tools like pandas for processing. [2]

Here is a Python block using **FastAPI** and **Pandas** to process the incoming Logic Apps payload:

```
import base64
import io
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import pandas as pd

app = FastAPI()

# 1. Define the schema matching your Logic App HTTP body
class LogicAppPayload(BaseModel):
    fileName: str
    fileContentBase64: str

@app.post("/api/agent/process")
async def process_excel_agent(payload: LogicAppPayload):
    try:
        # 2. Decode the Base64 string back into binary bytes
        file_bytes = base64.b64decode(payload.fileContentBase64)

        # 3. Read directly into memory using an In-Memory Bytes IO
        excel_stream = io.BytesIO(file_bytes)

        # 4. Use Pandas to load the Excel data
        df = pd.read_excel(excel_stream)

        # --- AT THIS POINT, YOUR AGENT HAS THE DATA ---
        # Convert to text or JSON so your LLM/Agent reasoning
        engine can read it
        excel_as_text = df.to_string()

        # Example Agent processing (e.g., passing to Azure OpenAI)
        agent_response = execute_agent_reasoning(excel_as_text)

    return {"status": "success", "agent_output":
agent_response}
```

```
except Exception as e:
    raise HTTPException(status_code=400, detail=f"Failed to
process file: {str(e)}")

def execute_agent_reasoning(data_context: str):
    # Your LangGraph / Semantic Kernel / OpenAI logic goes here
    return "Agent analysis of spreadsheet completed successfully."
```



Alternative Strategy for Large Files (>10MB)

If your Excel files are quite large, translating them into Base64 inside a JSON payload can hit Logic App or API payload size limitations. If that becomes an issue, use the **SAS URL pattern**:

1. In Logic Apps, instead of reading the file content, generate a **SAS URL (Shared Access Signature)** for the file in Azure Blob Storage.
2. Send a small JSON payload to your agent containing just the URL: {"fileUrl": "<sas_url_here>"}
3. Your agent code reads the URL directly using `pd.read_excel(payload.fileUrl)`.

Would you like a sample of **how your agent can pass this extracted Excel data to an Azure OpenAI model**, or are you looking for the specific code to package this into a container for Foundry deployment? [2, 7]

[1] <https://techcommunity.microsoft.com>

[2] <https://learn.microsoft.com>

[3] <https://www.linkedin.com>

[4] <https://learn.microsoft.com>

[5] <https://blog.chiffers.com>

[6] <https://stackoverflow.com>

[7] <https://techcommunity.microsoft.com>